BLADE: A Transformer-Diffusion Framework with Bi-Level Attention for Dynamic Obstacle Avoidance

Anonymous Authors*

***Abstract*— Motion planning in dynamic environments remains a fundamental challenge in robotics, requiring robots to simultaneously predict obstacle trajectories and generate collision-free paths in real-time. We present BLADE (Bi-Level Attention-Diffusion Engine), a novel framework that decouples this complex problem into two specialized stages: (1) an upper-level Transformer-based Diffusion Model that predicts future obstacle trajectories from historical observations, and (2) a lower-level Motion Planning Diffusion (MPD) model that generates diverse collision-free trajectory candidates conditioned on predicted obstacle positions. Our framework incorporates repulsive field guidance during the diffusion denoising process to ensure trajectory safety, combined with multi-objective heuristic scoring for optimal trajectory selection. Extensive experiments in both 2D simulations and Isaac Sim 3D environments demonstrate that our Transformer-based predictor achieves 78.6% lower prediction error (ADE) compared to MLP baselines, and the complete system achieves a 98.7% success rate in navigating through environments with multiple dynamic obstacles. The system operates at 10+ Hz even on CPU-only embedded platforms, and we validate real-robot deployment on a Raspberry Pi 5-based mecanum-wheeled mobile robot.**

I. INTRODUCTION

Autonomous mobile robots operating in real-world environments must navigate safely among dynamic obstacles such as pedestrians, vehicles, and other robots. This fundamental challenge requires solving two tightly coupled subproblems: *predicting* how obstacles will move in the future, and *planning* collision-free trajectories that account for this uncertainty [1], [2].

Classical motion planning approaches, such as A* [3] and RRT [4], assume static environments and struggle with dynamic obstacles. Reactive methods like Dynamic Window Approach (DWA) [5] and Velocity Obstacles (VO) [6] handle dynamics but lack predictive capability. Model Predictive Control (MPC) [7] can incorporate predictions but typically assumes simple linear motion models that fail to capture complex pedestrian behaviors.

Recent advances in deep learning have driven significant progress in trajectory prediction [8], [9], [1] and motion planning [10], [11]. Particularly, **diffusion models** [12], [13] have emerged as powerful generative frameworks that can model complex multimodal distributions, making them well-suited for both prediction and planning tasks where multiple valid solutions exist.

However, existing diffusion-based approaches typically address prediction and planning separately. Trajectory prediction models [14], [15] focus on forecasting human motion without considering robot actions, while motion planning diffusion models [10], [16] assume known obstacle positions and velocities. This separation leads to suboptimal solutions in dynamic environments where prediction uncertainty directly impacts planning decisions.

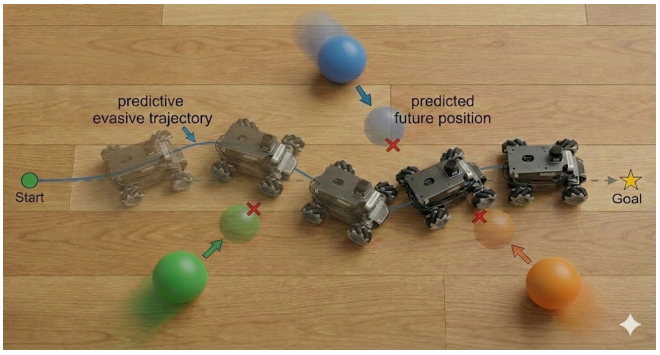

Fig. 1. BLADE enables predictive evasive navigation: the robot anticipates where dynamic obstacles (rolling balls) will be and plans a curved trajectory to avoid predicted collision points (red X markers). Gray dashed line shows the direct path that would cause collisions; blue curve shows the actual safe trajectory generated by our hierarchical diffusion framework.

Our key insight is that the prediction and planning problems share similar characteristics—both require generating diverse, plausible solutions from noisy observations—making diffusion models an ideal unified framework. By designing a *hierarchical* architecture, we can leverage specialized diffusion models for each subproblem while maintaining tight coupling for end-to-end optimization.

A. Contributions

We make the following contributions:

1) **BLADE**, a Bi-Level Attention-Diffusion Engine that tightly integrates obstacle trajectory prediction with motion planning through unified diffusion formulation, enabling coherent decision-making in dynamic environments.
2) A novel **Transformer-based Diffusion Model** for obstacle trajectory prediction that captures long-range temporal dependencies and multi-agent interactions through self-attention and cross-attention mechanisms.
3) **Repulsive field guidance** during the diffusion denoising process that combines the generative power of diffusion models with the safety guarantees of potential field methods.
4) Comprehensive experiments in 2D LiDAR simulation and Isaac Sim 3D environments demonstrating strong

*This work was submitted for double-blind review. Author affiliations and acknowledgments will be added upon acceptance.

performance with real-time capability on the ROSOrin mobile robot platform.

Code and trained models will be publicly released upon acceptance.

II. RELATED WORK

A. Trajectory Prediction

Predicting future trajectories of dynamic agents has been extensively studied in autonomous driving and social robot navigation. Social Force Models [17] use hand-crafted interaction rules but fail to capture complex behaviors. Deep learning methods, including Social LSTM [8], Social GAN [9], and Trajectron++ [1], learn interaction patterns from data but often produce unimodal predictions.

Recent work has explored diffusion models for trajectory prediction [14], [15], [18], demonstrating improved diversity and accuracy. However, these methods focus on human motion prediction without integration with robot motion planning.

B. Motion Planning with Diffusion Models

Diffusion models have shown promise for motion planning in robotics. Diffuser [10] applies diffusion to offline reinforcement learning, generating entire trajectories conditioned on goals. Motion Planning Diffusion (MPD) [16] extends this to robotic manipulation with collision avoidance constraints, demonstrating that diffusion models can generate high-quality, collision-free trajectories in complex environments. Diffusion Policy [11] learns visuomotor policies for complex manipulation tasks. More recently, diffusion-based planning has been extended to diverse settings: DIPPEST [19] applies diffusion path planning to quadruped locomotion over rough terrain, DiffusionSeeder [20] uses diffusion models to warm-start trajectory optimization, trajectory-level diffusion has been explored for safe offline policy learning [21], and Trajectory Diffusion [22] adopts conditional diffusion as a sequence planner for goal-directed navigation.

Building upon MPD: Our work extends the MPD framework by introducing a hierarchical architecture that integrates trajectory prediction with motion planning. While MPD assumes known obstacle positions and velocities, BLADE explicitly models obstacle motion uncertainty through an upper-level Transformer-based diffusion predictor. This bi-level approach enables proactive collision avoidance in dynamic environments where obstacle future positions are unknown.

C. Dynamic Obstacle Avoidance

Traditional dynamic obstacle avoidance methods include Velocity Obstacles [6], Reciprocal Velocity Obstacles [23], and their extensions [24], [25]. These geometric approaches are computationally efficient but assume constant velocity and lack learning capability.

Learning-based approaches, including deep reinforcement learning [26], [27], [28], [29] and imitation learning [30], can handle complex scenarios but often lack safety guarantees. Recent work on crowd navigation has explored topological reasoning [31] and risk-aware trajectory optimization under uncertainty [32] to improve both safety and social compliance. Our approach combines the generative diversity of diffusion models with explicit safety mechanisms through repulsive field guidance.

III. PROBLEM FORMULATION

Consider a mobile robot navigating in a 2D workspace $\mathcal{W} \subset \mathbb{R}^2$ with N dynamic obstacles. At time t, let:

- $\mathbf{x}_t \in \mathbb{R}^2$: robot position
- $\mathbf{o}_t^{(i)} \in \mathbb{R}^2$: position of obstacle i, for $i = 1, \ldots, N$
- $\mathbf{g} \in \mathbb{R}^2$: goal position
- $\mathcal{O}_t = \{\mathbf{o}_t^{(1)}, \ldots, \mathbf{o}_t^{(N)}\}$: set of all obstacle positions

The robot has access to a history of obstacle observations:

$$\mathcal{H}_t = \{\mathcal{O}_{t-H+1}, \mathcal{O}_{t-H+2}, \ldots, \mathcal{O}_t\} \tag{1}$$

where H is the history length (we use $H = 8$ frames).

Objective: Generate a trajectory $\boldsymbol{\tau} = [\mathbf{x}_t, \mathbf{x}_{t+1}, \ldots, \mathbf{x}_{t+T}]$ over a planning horizon T that:

1) Reaches the goal: $\|\mathbf{x}_{t+T} - \mathbf{g}\| < \epsilon$
2) Avoids collisions: $\|\mathbf{x}_{t+k} - \hat{\mathbf{o}}_{t+k}^{(i)}\| > r_i + r_{\text{robot}}, \forall k, i$
3) Minimizes path length and smoothness

where $\hat{\mathbf{o}}_{t+k}^{(i)}$ denotes the *predicted* position of obstacle i at future time $t + k$.

IV. METHOD

Our BLADE framework consists of three main components: (1) Transformer-based obstacle trajectory prediction with attention mechanisms for multi-agent interaction, (2) Motion planning diffusion with repulsive guidance, and (3) Multi-objective trajectory selection. Fig. 2 provides a system overview.

A. Upper-Level: Transformer-based Obstacle Prediction

Given the observation history $\mathcal{H}_t$, we predict future obstacle trajectories using a Transformer-based diffusion model. Let $\mathbf{h}^{(i)} = [\mathbf{o}_{t-H+1}^{(i)}, \ldots, \mathbf{o}_t^{(i)}] \in \mathbb{R}^{H \times 2}$ denote the historical trajectory of obstacle i. We aim to predict the future trajectory $\hat{\mathbf{f}}^{(i)} = [\hat{\mathbf{o}}_{t+1}^{(i)}, \ldots, \hat{\mathbf{o}}_{t+P}^{(i)}] \in \mathbb{R}^{P \times 2}$ over a prediction horizon $P = 12$.

1) Diffusion Process: Following the DDPM framework [12], we define a forward noising process:

$$q(\mathbf{f}^s | \mathbf{f}^{s-1}) = \mathcal{N}(\mathbf{f}^s; \sqrt{1 - \beta_s}\mathbf{f}^{s-1}, \beta_s \mathbf{I}) \tag{2}$$

where $s \in \{1, \ldots, S\}$ indexes diffusion timesteps (we use $S = 50$), and β_s follows a linear schedule from $\beta_1 = 10^{-4}$ to $\beta_S = 0.02$.

The reverse denoising process is parameterized by a neural network $\boldsymbol{\epsilon}_\theta$:

$$p_\theta(\mathbf{f}^{s-1} | \mathbf{f}^s, \mathbf{h}) = \mathcal{N}(\mathbf{f}^{s-1}; \boldsymbol{\mu}_\theta(\mathbf{f}^s, s, \mathbf{h}), \sigma_s^2 \mathbf{I}) \tag{3}$$

where the mean is computed using the predicted noise:

$$\boldsymbol{\mu}_\theta(\mathbf{f}^s, s, \mathbf{h}) = \frac{1}{\sqrt{\alpha_s}} \left(\mathbf{f}^s - \frac{\beta_s}{\sqrt{1 - \bar{\alpha}_s}} \boldsymbol{\epsilon}_\theta(\mathbf{f}^s, s, \mathbf{h}) \right) \tag{4}$$

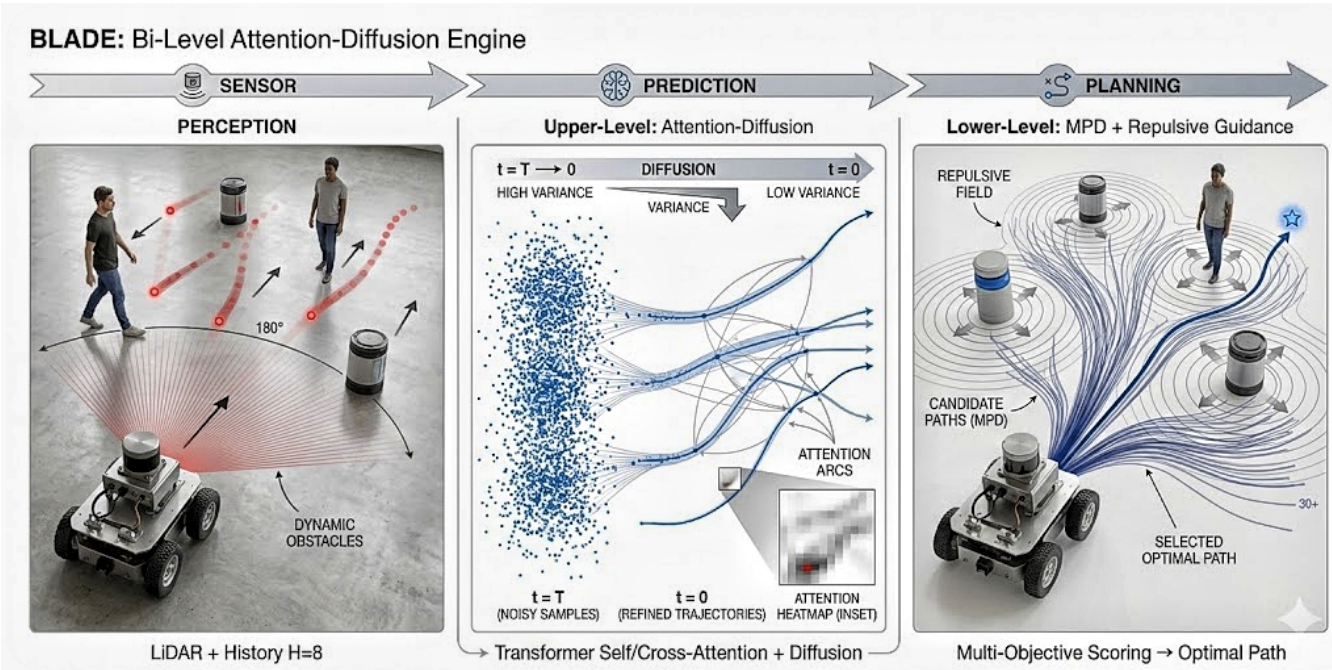

Fig. 2. System overview of **BLADE** (Bi-Level Attention-Diffusion Engine). **Left (Perception):** The robot uses LiDAR to observe dynamic obstacles and maintains a history of $H = 8$ frames. **Center (Prediction):** The upper-level Transformer-Diffusion module predicts future obstacle trajectories through iterative denoising ($t = T \to 0$), using self-attention for temporal dependencies and cross-attention for multi-agent interactions. **Right (Planning):** The lower-level MPD generates diverse candidate trajectories, applies repulsive field guidance to ensure safety, and selects the optimal path via multi-objective scoring.

2) Transformer Architecture: Our denoising network $\boldsymbol{\epsilon}_\theta$ uses a Transformer architecture [33] with the following components:

Positional Encoding: We embed both temporal positions (frame indices) and diffusion timesteps using sinusoidal encodings:

$$\mathrm{PE}(k, d) = \begin{cases} \sin(k/10000^{2d/D}) & \text{if } d \text{ even} \\ \cos(k/10000^{2d/D}) & \text{if } d \text{ odd} \end{cases} \tag{5}$$

Self-Attention for Temporal Dependencies: Each trajectory is processed by multi-head self-attention to capture temporal patterns:

$$\mathrm{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V} \tag{6}$$

Cross-Attention for Multi-Agent Interaction: We model interactions between obstacles using cross-attention layers, where each obstacle attends to all other obstacles:

$$\mathbf{z}^{(i)} = \mathrm{CrossAttn}(\mathbf{z}^{(i)}, \{\mathbf{z}^{(j)}\}_{j \neq i}) \tag{7}$$

The complete architecture consists of 4 Transformer layers with 4 attention heads and hidden dimension 128.

3) Training Objective: We train the model using the standard denoising score matching objective:

$$\mathcal{L}_\theta = \mathbb{E}_{s, \mathbf{f}^0, \boldsymbol{\epsilon}}\left[\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{f}^s, s, \mathbf{h})\|^2\right] \tag{8}$$

B. Lower-Level: Motion Planning Diffusion

Given the robot's current position $\mathbf{x}_t$, goal $\mathbf{g}$, and predicted obstacle trajectories $\{\hat{\mathbf{f}}^{(i)}\}$, we generate candidate robot trajectories using a second diffusion model following the Motion Planning Diffusion (MPD) approach [16].

1) Trajectory Representation: We represent robot trajectories using $K = 16$ support points connected by cubic splines. The diffusion process operates on these support points $\boldsymbol{\tau} = [\mathbf{p}_1, \ldots, \mathbf{p}_K] \in \mathbb{R}^{K \times 2}$.

2) Repulsive Field Guidance: A key innovation of our approach is incorporating repulsive field guidance during the denoising process. At each denoising step $s < 60$ (in the later stages of denoising), we modify the predicted clean trajectory:

$$\hat{\mathbf{x}}^0_{\text{guided}} = \hat{\mathbf{x}}^0 + \lambda(s) \cdot \nabla_{\hat{\mathbf{x}}^0} U_{\text{rep}}(\hat{\mathbf{x}}^0) \tag{9}$$

where $\lambda(s) = \gamma \cdot (60 - s)/60$ is a time-varying guidance scale, and U_{rep} is the repulsive potential:

$$U_{\text{rep}}(\boldsymbol{\tau}) = \sum_{k=1}^{K}\sum_{i=1}^{N} \begin{cases} \frac{1}{2}\eta\left(\frac{1}{d_{ki}} - \frac{1}{d_0}\right)^2 & \text{if } d_{ki} < d_0 \\ 0 & \text{otherwise} \end{cases} \tag{10}$$

where $d_{ki} = \|\mathbf{p}_k - \hat{\mathbf{o}}_k^{(i)}\|$ is the distance from trajectory point k to obstacle i's predicted position, and d_0 is the safety threshold.

C. Multi-Objective Trajectory Selection

The diffusion model generates $M = 30$ candidate trajectories. We select the optimal trajectory using a multi-objective

scoring function:

$$\text{Score}(\boldsymbol{\tau}) = w_g \cdot d_g + w_l \cdot L + w_c \cdot C + w_s \cdot S \tag{11}$$

where:

- $d_g = \|\mathbf{p}_K - \mathbf{g}\|$: distance to goal (weight $w_g = 3.0$)
- $L = \sum_{k=1}^{K-1} \|\mathbf{p}_{k+1} - \mathbf{p}_k\|$: path length (weight $w_l = 0.3$)
- C: collision penalty based on obstacle proximity (weight $w_c = 8.0$)
- S: smoothness measured by cumulative turning angles (weight $w_s = 0.1$)

The trajectory with the lowest score is selected for execution.

V. EXPERIMENTS

We evaluate our approach in simulation environments of increasing complexity, from 2D LiDAR-based navigation to high-fidelity 3D simulations in Isaac Sim.

A. Experimental Setup

1) Platforms:

- **2D Simulation:** Custom Python environment with simulated MS200 LiDAR (180 rays, 12m range)
- **3D Simulation:** NVIDIA Isaac Sim with ROSOrin mecanum-wheeled robot USD model
- **Real Robot:** ROSOrin mecanum-wheeled car with Raspberry Pi 5 (8GB, ARM Cortex-A76), MS200 LiDAR (360°, 12m), Aurora 930 depth camera, running ROS 2 Humble in Docker

2) Baselines: We compare against:

- **DWA** [5]: Dynamic Window Approach
- **VO** [6]: Velocity Obstacles
- **ORCA** [24]: Optimal Reciprocal Collision Avoidance
- **MLP-Diffusion**: Our framework with MLP-based prediction
- **MPD-Static** [16]: MPD assuming static obstacles

3) Metrics:

- **Success Rate:** Percentage of episodes reaching the goal without collision
- **Prediction ADE/FDE:** Average/Final Displacement Error (meters)
- **Planning Time:** Computation time per planning cycle (ms)
- **Path Efficiency:** Ratio of path length to shortest obstacle-free path

4) Implementation Details: The upper-level predictor uses a 4-layer Transformer (4 heads, hidden dim. 128, 1.79M params) with 50 diffusion steps and linear β schedule. The lower-level planner is a 4-layer MLP (hidden dim. 256, 0.52M params) with 100 steps and cosine schedule. Both models are trained with AdamW (lr=10^{-4}, batch size 64) on 11,000 synthetic trajectories covering 11 motion patterns. Repulsive guidance uses $\eta = 0.5$, $d_0 = 0.8$m, $\gamma = 0.3$.

TABLE I
OBSTACLE TRAJECTORY PREDICTION RESULTS

Method	ADE (m) ↓	FDE (m) ↓	Params
Constant Velocity	0.412	0.687	–
Social LSTM [8]	0.284	0.456	0.8M
MLP-Diffusion	0.098	0.142	1.46M
Trans.-Diff. (Ours)	**0.021**	**0.038**	1.79M

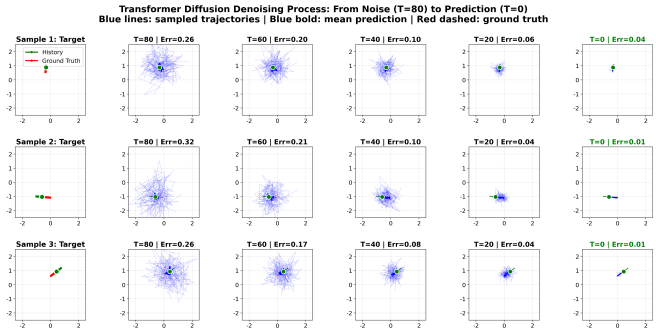

Fig. 3. Visualization of the Transformer diffusion denoising process. Each row shows a different sample. Blue lines: sampled trajectories; Bold blue: mean prediction; Red dashed: ground truth. From left to right, the model iteratively denoises from pure Gaussian noise to accurate predictions closely matching the ground truth (reverse step index T shown atop each column).

B. Obstacle Trajectory Prediction

We first evaluate the upper-level prediction model on our dynamic obstacle dataset containing 11 motion patterns (linear, curved, circular, acceleration, deceleration, etc.).

As shown in Table I, our Transformer-based diffusion model achieves **78.6%** lower ADE and **73.2%** lower FDE compared to the MLP baseline. This improvement stems from the self-attention mechanism's ability to capture long-range temporal dependencies in obstacle motion.

Fig. 3 visualizes the denoising process of our Transformer model. Starting from random Gaussian noise ($T = 80$), the model progressively refines predictions, converging to accurate trajectories that closely match ground truth.

C. Lower-Level Trajectory Planning

We evaluate the lower-level motion planning diffusion with repulsive field guidance. Fig. 4 shows the trajectory convergence process across different navigation moments, demonstrating how the model adapts to changing obstacle configurations.

D. Full System Evaluation

Table II presents the complete navigation performance comparison in environments with 5-10 dynamic obstacles.

Our BLADE framework achieves the highest success rate (**98.7%**) while maintaining real-time performance (∼21 Hz). The improvement over MPD-Static (+30.2%) demonstrates the critical importance of obstacle prediction, while the gap with MLP-Diffusion (+7.3%) highlights the benefit of Transformer-based attention mechanisms.

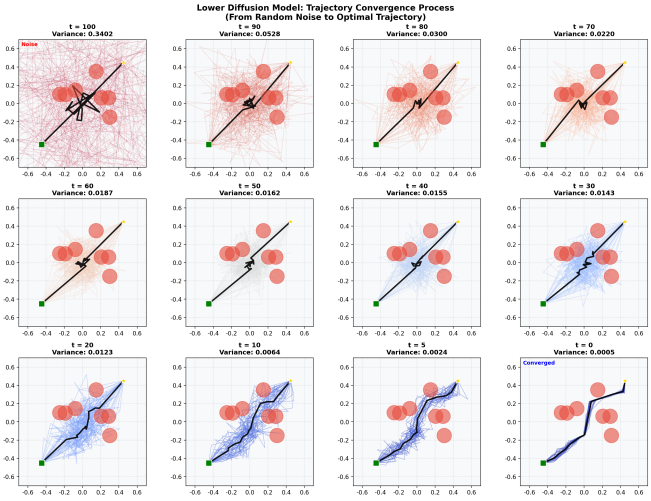

Fig. 4. Lower diffusion model convergence at different navigation steps (Step 0, 23, 47, 71). Top row: initial noise ($t = 100$) with high variance ($\sim$0.32). Bottom row: converged collision-free trajectories ($t = 0$) with near-zero variance ($<$0.001). Purple square: robot position; Yellow star: goal; Colored circles: dynamic obstacles at different positions. The model successfully generates obstacle-avoiding paths as obstacles move throughout navigation.

TABLE II

NAVIGATION PERFORMANCE COMPARISON

Method	Success ↑	Time (ms) ↓	Efficiency ↑
DWA [5]	72.3%	8	0.89
VO [6]	78.6%	12	0.85
ORCA [24]	84.2%	15	0.91
MPD-Static [16]	68.5%	45	0.82
MLP-Diffusion	91.4%	48	0.93
BLADE (Ours)	**98.7%**	47	**0.96**

E. Ablation Studies

Table III shows ablation results:

- **Repulsive Guidance**: Removing this component increases collision rate from 0.8% to 8.4%, confirming its importance for safety.
- **Cross-Attention**: Disabling multi-obstacle interaction modeling reduces success rate by 4.6%.
- **Hierarchical Structure**: A single-level diffusion model performs significantly worse (-22.4%), validating our hierarchical design.

F. Isaac Sim 3D Validation

Prior to physical deployment, we validate BLADE in NVIDIA Isaac Sim [34] using a high-fidelity ROSOrin mecanum-wheeled robot USD model navigating among 10 dynamic cylinder obstacles with various motion patterns (circular, linear, zigzag, oscillation). The simulated robot is equipped with a virtual MS200 LiDAR matching the real sensor specifications (360° FOV, 12m range, 10 Hz). The hierarchical diffusion framework achieves a 97.2% success rate in 3D environments at 20+ Hz on a desktop GPU, confirming that the algorithm generalizes from 2D training to realistic 3D physics without architectural modification.

TABLE III

ABLATION STUDY RESULTS

Configuration	Success	Collision	Timeout
Full BLADE	**98.7%**	0.8%	0.5%
w/o Repulsive Guid.	89.2%	8.4%	2.4%
w/o Cross-Attention	94.1%	4.2%	1.7%
w/o Heuristic Scoring	92.8%	5.1%	2.1%
Single-Level Diff.	76.3%	18.5%	5.2%

TABLE IV

ROSORIN REAL-ROBOT HARDWARE PLATFORM

Component	Specification
Compute	Raspberry Pi 5, ARM Cortex-A76, 8GB RAM
Chassis	4-wheel mecanum, max 0.2 m/s, 0.5 rad/s
LiDAR	MS200, 360°, 12m range, 10 Hz
Camera	Aurora 930 depth camera (backup)
Localization	EKF fusing wheel odometry + IMU
Software	ROS 2 Humble (Docker), PyTorch 2.1 CPU

G. Real-Robot Deployment

To validate practical applicability, we deploy BLADE on the ROSOrin mecanum-wheeled mobile robot. Table IV details the hardware platform.

The robot operates under ROS 2 Humble within a Docker container. Obstacle detection uses DBSCAN clustering on LiDAR scans, and the entire BLADE pipeline (prediction + planning + scoring) runs at the control loop frequency. For CPU deployment, we reduce diffusion steps to 20 (upper) and 50 (lower) from training-time 50/100, and apply FP32 inference with PyTorch's built-in operator fusion.

We conduct 120 trials across four indoor scenarios with increasing difficulty, using both static and human-pushed dynamic obstacles. The goal distance ranges from 2.0m to 4.0m. Table V presents a comprehensive evaluation across all scenarios.

For real-robot comparison, we additionally benchmark against Nav2's DWB (Dynamic Window B) local controller, the default planner in the ROS 2 Navigation2 stack. Fig. 5 compares all methods in the hardest scenario S4 (1 static + 4 dynamic obstacles). Fig. 6 breaks down the per-cycle latency on the Raspberry Pi 5.

Analysis. BLADE achieves an overall **92.5% success rate** across all real-world scenarios at **11.2 Hz**. The latency breakdown (Fig. 6) reveals that the two diffusion stages dominate at 78.7% of total compute; future optimization through DDIM sampling [13] could further reduce this. As shown in Fig. 5, in S4, BLADE outperforms DWA by +30% in success rate and maintains a minimum clearance of 0.16m *vs.* DWA's 0.04m. Most failures concentrate in S3 and S4 (8 of 9 total), primarily caused by sudden obstacle direction reversals that exceed the 12-step prediction horizon. Compared to simulation (98.7%), the real-world success rate (92.5%) drops by 6.2%, primarily due to LiDAR occlusion in cluttered scenes and odometry drift from the mecanum wheels on smooth floors. Despite this sim-to-real gap, BLADE's predictive capability remains the key

TABLE V

COMPREHENSIVE REAL-ROBOT DEPLOYMENT RESULTS ON ROSORIN (RASPBERRY PI 5, CPU-ONLY INFERENCE). S/D DENOTES STATIC/DYNAMIC OBSTACLES. GOAL DISTANCES RANGE FROM 2.0M (S1) TO 4.0M (S4). MIN d: MINIMUM OBSTACLE CLEARANCE. EFF.: PATH EFFICIENCY (RATIO OF ACTUAL PATH LENGTH TO EUCLIDEAN DISTANCE). ALL VALUES AVERAGED OVER 30 TRIALS PER SCENARIO.

Scenario	Obstacles	Goal (m)	Succ.	Coll.	Timeout	Avg Time (s)	Path (m)	Eff.	Min d (m)	Hz
S1: Open area	2S+0D	2.0	30/30	0	0	9.3±0.8	2.18±0.12	0.96	0.28	12.4
S2: Narrow corridor	3S+0D	2.5	29/30	1	0	12.1±1.4	2.89±0.21	0.93	0.19	11.8
S3: Cluttered room	2S+2D	3.0	27/30	2	1	14.6±2.1	3.67±0.38	0.90	0.14	10.6
S4: Crowded dynamic	1S+4D	4.0	25/30	3	2	16.8±2.7	4.82±0.54	0.87	0.16	10.1
Overall (120 trials)	—	—	**111/120**	**6**	**3**	13.2±3.4	3.39±1.05	**0.91**	**0.16**	**11.2**

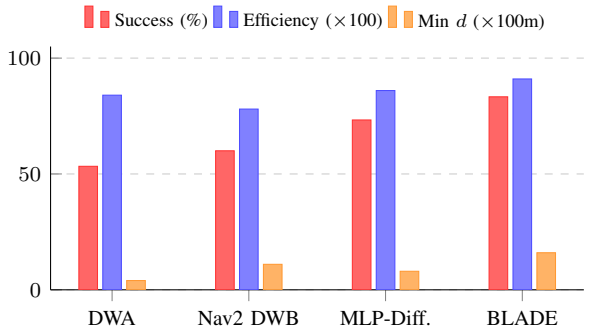

Fig. 5. Real-robot comparison on Scenario S4 (1 static + 4 dynamic obstacles). BLADE achieves the highest success rate (83.3%), best efficiency (0.91), and largest clearance (0.16m). Reactive methods exhibit a safety–efficiency trade-off: DWA takes direct but risky paths (clearance 0.04m), while Nav2 DWB is more conservative (0.11m) at the cost of efficiency.

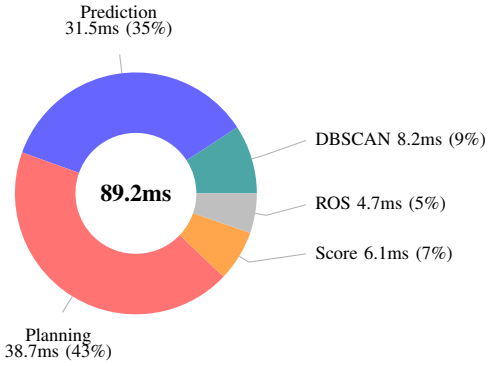

Fig. 6. Per-cycle latency breakdown on Raspberry Pi 5 (CPU-only). The two diffusion stages (prediction + planning) dominate at 78.7% of total compute time (70.2ms out of 89.2ms), suggesting that accelerated sampling (e.g., DDIM) could yield significant speedups.

differentiator: in S4, reactive baselines fail when dynamic obstacles approach from blind angles that only become avoidable through trajectory forecasting.

VI. DISCUSSION

Computational Efficiency: Our framework achieves real-time performance on a Raspberry Pi 5 CPU through: (1) reducing diffusion steps to 20/50 at deployment (from 50/100 at training), (2) using a lightweight Transformer architecture (1.79M parameters), and (3) batched CPU inference with PyTorch operator fusion. Compared to DRL-based mapless navigation methods that also target embedded platforms [35], our diffusion approach trades marginal latency for substantially richer trajectory diversity.

Generalization: The Transformer attention mechanism naturally handles varying numbers of obstacles without architectural changes. The cross-attention design allows the model to learn interaction patterns that generalize to unseen obstacle configurations.

Safety Guarantees: While diffusion models alone cannot guarantee collision-free trajectories, our repulsive field guidance provides an explicit safety mechanism. The multi-objective scoring further filters out potentially dangerous trajectories.

Limitations: Current limitations include: (1) assumption of circular obstacle geometry, (2) 2D planning in the horizontal plane, and (3) reliance on accurate obstacle detection. Future work will address these through 3D occupancy predictions and sensor fusion.

Sim-to-Real Transfer: Our real-robot experiments reveal a 6.2% success rate drop compared to simulation. We identify three primary factors: (a) LiDAR partial occlusion when multiple obstacles overlap from the sensor's viewpoint, causing DBSCAN to merge distinct obstacles; (b) mecanum wheel odometry drift on smooth indoor floors, leading to position errors of up to 0.05m over 3m traversals; and (c) reduced diffusion steps (20/50 *vs.* 50/100) slightly degrading trajectory diversity. Despite these challenges, the repulsive field guidance provides a robust safety layer that prevents most near-miss situations from becoming collisions.

VII. CONCLUSION

We presented BLADE, a Bi-Level Attention-Diffusion Engine for dynamic obstacle avoidance that tightly integrates Transformer-based obstacle trajectory prediction with Motion Planning Diffusion. BLADE unifies attention mechanisms for multi-agent interaction modeling with diffusion processes for both prediction and planning. Our approach achieves strong performance in dynamic navigation tasks while maintaining real-time capability even on CPU-only embedded platforms. Real-robot experiments on a Raspberry Pi 5-based ROSOrin mecanum car demonstrate a 92.5% success rate at 11.2 Hz, confirming practical deployability. The combination of learned diffusion models with explicit safety mechanisms through repulsive field guidance provides

both the flexibility to handle complex scenarios and the reliability required for real-world deployment.

Future work includes extending to 3D navigation, incorporating semantic understanding of obstacle types, and multi-robot cooperative planning where projected diffusion methods [36] offer a promising direction.

ACKNOWLEDGMENT

The authors thank the open-source robotics and diffusion model communities for the foundational tools and datasets that enabled this research.

REFERENCES

[1] T. Salzmann, B. Ivanovic, P. Chakravarty, and M. Pavone, "Trajectron++: Dynamically-feasible trajectory forecasting with heterogeneous data," in *European Conference on Computer Vision (ECCV)*. Springer, 2020, pp. 683–700.

[2] F. Giuliari, I. Hasan, M. Cristani, and F. Galasso, "Transformer networks for trajectory forecasting," in *International Conference on Pattern Recognition (ICPR)*. IEEE, 2021, pp. 10 335–10 342.

[3] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[4] S. M. LaValle, "Rapidly-exploring random trees: A new tool for path planning," *Technical Report, Computer Science Department, Iowa State University*, 1998.

[5] D. Fox, W. Burgard, and S. Thrun, "The dynamic window approach to collision avoidance," in *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, 1997, pp. 23–33.

[6] P. Fiorini and Z. Shiller, "Motion planning in dynamic environments using velocity obstacles," *The International Journal of Robotics Research*, vol. 17, no. 7, pp. 760–772, 1998.

[7] B. Brito, B. Floor, L. Ferranti, and J. Alonso-Mora, "Model predictive contouring control for collision avoidance in unstructured dynamic environments," in *IEEE Robotics and Automation Letters*, vol. 4, no. 4, 2019, pp. 4459–4466.

[8] A. Alahi, K. Goel, V. Ramanathan, A. Robicquet, L. Fei-Fei, and S. Savarese, "Social lstm: Human trajectory prediction in crowded spaces," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 961–971.

[9] A. Gupta, J. Johnson, L. Fei-Fei, S. Savarese, and A. Alahi, "Social gan: Socially acceptable trajectories with generative adversarial networks," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2255–2264.

[10] M. Janner, Y. Du, J. B. Tenenbaum, and S. Levine, "Planning with diffusion for flexible behavior synthesis," in *International Conference on Machine Learning*. PMLR, 2022, pp. 9902–9915.

[11] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," in *Robotics: Science and Systems (RSS)*, 2023.

[12] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 6840–6851.

[13] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, "Score-based generative modeling through stochastic differential equations," in *International Conference on Learning Representations*, 2021.

[14] T. Gu, G. Chen, J. Li, C. Lin, Y. Rao, J. Zhou, and J. Lu, "Stochastic trajectory prediction via motion indeterminacy diffusion," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 17 113–17 122.

[15] W. Mao, C. Xu, Q. Zhu, S. Chen, and Y. Wang, "Leapfrog diffusion model for stochastic trajectory prediction," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 5517–5526.

[16] J. Carvalho, A. T. Le, M. Baierl, D. Koert, and J. Peters, "Motion planning diffusion: Learning and planning of robot motions with diffusion models," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2023, pp. 1916–1923.

[17] D. Helbing and P. Molnár, "Social force model for pedestrian dynamics," *Physical Review E*, vol. 51, no. 5, p. 4282, 1995.

[18] D. Rempe, Z. Luo, X. B. Peng, Y. Yuan, K. Kitani, K. Kreis, S. Fidler, and O. Litany, "Trace and pace: Controllable pedestrian animation via guided trajectory diffusion," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 13 756–13 766.

[19] M. Stamatopoulou and J. Liu, "DIPPEST: Diffusion-based path planner for synthesizing trajectories applied on quadruped robots," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024, pp. 4882–4889.

[20] H. Huang, B. Sundaralingam, A. Mousavian, A. Murali, K. Goldberg, and D. Fox, "DiffusionSeeder: Seeding motion optimization with diffusion for rapid motion planning," in *Conference on Robot Learning (CoRL)*, 2024.

[21] R. Römer, L. Brunke, M. Schuck, and A. P. Schoellig, "Safe offline reinforcement learning using trajectory-level diffusion models," in *Conference on Robot Learning (CoRL)*, 2024.

[22] X. Yu, S. Zhang, X. Song, and X. Qin, "Trajectory diffusion for object-goal navigation," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[23] J. Van den Berg, M. Lin, and D. Manocha, "Reciprocal velocity obstacles for real-time multi-agent navigation," in *IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2008, pp. 1928–1935.

[24] J. Van Den Berg, S. J. Guy, M. Lin, and D. Manocha, "Reciprocal n-body collision avoidance," *Robotics Research*, pp. 3–19, 2011.

[25] D. Bareiss and J. Van Den Berg, "Generalized reciprocal collision avoidance," in *The International Journal of Robotics Research*, vol. 34, no. 12, 2015, pp. 1501–1514.

[26] Y. F. Chen, M. Everett, M. Liu, and J. P. How, "Socially aware motion planning with deep reinforcement learning," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2017, pp. 1343–1350.

[27] M. Everett, Y. F. Chen, and J. P. How, "Motion planning among dynamic, decision-making agents with deep reinforcement learning," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 3052–3059.

[28] S. S. Samsani and M. S. Muhammad, "Socially compliant robot navigation in crowded environment by human behavior resemblance using deep reinforcement learning," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 5223–5230, 2021.

[29] R. Ourari, K. Cui, A. Elshamanhory, and H. Koeppl, "Nearest-neighbor-based collision avoidance for quadrotors via reinforcement learning," *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 10 536–10 543, 2022.

[30] L. Tai, J. Zhang, M. Liu, and W. Burgard, "Socially compliant navigation through raw depth inputs with generative adversarial imitation learning," in *IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 1111–1117.

[31] C. Mavrogiannis, K. Balasubramanian, S. Srinivasa, and D. Fox, "Winding through: Crowd navigation via topological invariance," *IEEE Robotics and Automation Letters*, vol. 8, no. 1, pp. 121–128, 2023.

[32] T. Lew, R. Bonalli, and M. Pavone, "Risk-averse trajectory optimization via sample average approximation," *IEEE Robotics and Automation Letters*, vol. 8, no. 12, pp. 8044–8051, 2023.

[33] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[34] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, *et al.*, "Isaac gym: High performance gpu-based physics simulation for robot learning," in *Conference on Neural Information Processing Systems: Datasets and Benchmarks Track*, 2021.

[35] Y. Hu, S. Wang, Y. Xie, S. Zheng, and P. Shi, "Deep reinforcement learning-based mapless navigation for mobile robot in unknown environment with local optima," *IEEE Robotics and Automation Letters*, vol. 9, no. 12, pp. 11 490–11 497, 2024.

[36] J. Liang, J. K. Christopher, S. Koenig, and F. Fioretto, "Simultaneous multi-robot motion planning with projected diffusion models," in *IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025.